

以下为您总结的关于**文件系统 (File System)**、**主存与虚拟内存 (Main Memory & Virtual Memory)** 以及**磁盘存储 (Mass Storage)** 相关的所有核心计算题解决方法。总结采用“中文 + 英文 + 简洁公式 + 历年真题参考”的格式，方便您考试时快速查阅。

一、虚拟内存与主存 (Virtual Memory & Main Memory)

1. 逻辑地址划分与物理地址转换 (Logical to Physical Address Translation)

- **计算目标:** 将逻辑地址 (Logical Address) 拆分为页号 (Page Number) 和页偏移 (Page Offset)，并映射到物理地址 (Physical Address)。
- **简洁公式:**
 - 页偏移位数 (Offset bits) = $\log_2(\text{页面大小 Page Size})$
 - 页号位数 (Page Number bits) = 逻辑地址总位数 - 页偏移位数
 - 物理地址 (Physical Address) = (物理帧号 Frame Number $\ll$ Offset bits) + 偏移量 Offset
- **参考真题:** 2025 Spring Q3.1, 2024 Fall Q3.3, 2025 Fall Q3.3

2. 多级页表大小与条目数计算 (Multi-level Page Table Size)

- **计算目标:** 计算为了映射一定大小的逻辑内存，页表需要占用多少实际物理内存。
- **简洁公式:**
 - 每个页表的条目数 (Entries per page) = $\frac{\text{页面大小 Page Size}}{\text{页表项大小 PTE Size}}$
 - 需要的总数据页数 (Data Pages) = $\frac{\text{进程逻辑空间大小 Allocated Space}}{\text{页面大小 Page Size}}$
 - 底层页表数 (Inner Page Tables) = $\lceil \frac{\text{Data Pages}}{\text{Entries per page}} \rceil$
 - 总页表大小 = (顶级页表数 + 底层页表数) $\times$ 页面大小
- **参考真题:** 2025 Spring Q3.1(2), 2024 Spring Q3.1, 2025 Fall Q3.1

3. 有效访问时间 (Effective Access Time, EAT)

- **计算目标:** 结合 TLB 命中率 (α)、内存访问时间 (T_{mem})、TLB 查找时间 (T_{tlb}) 和缺页率 (P_{fault})，计算平均访问时间。
- **简洁公式 (无缺页情况 / Basic):**
 - $EAT = \alpha \times (T_{tlb} + T_{mem}) + (1 - \alpha) \times (T_{tlb} + k \times T_{mem})$
 - (注: k 为多级页表需要访问内存的次数，例如单级页表 $k = 2$ ，二级页表 $k = 3$)
- **简洁公式 (含缺页情况 / With Page Fault):**
 - $EAT = P(\text{TLB Hit, No PF}) \times T_{hit} + P(\text{TLB Miss, No PF}) \times T_{miss} + P(\text{TLB Miss, PF}) \times T_{fault}$
 - (注: 通常 TLB Hit 时不会触发 Page Fault)
- **参考真题:** 2025 Spring Q3.1(2.3/2.4), 2025 Fall Q4.2, 2024 Fall Q1.MC11

4. 伙伴系统碎片计算 (Buddy System Fragmentation)

- **计算目标:** 计算经过一系列分配和释放后，伙伴系统中的外部碎片和内部碎片大小。
- **简洁公式:**
 - 外部碎片 (External Fragmentation) = $\sum \text{所有小于当前请求大小的空闲块 (Free Blocks) 容量}$

- 内部碎片 (Internal Fragmentation) = $\sum(\text{已分配块容量 Allocated Block Size} - \text{进程实际请求容量 Requested Size})$
- 参考真题: 2025 Fall Q3.4 (Buddy System 树形图与碎片计算)

二、文件系统 (File System)

1. Inode 最大文件大小支持 (Maximum File Size via Inode)

- 计算目标: 基于直接指针、单级/双级/三级间接指针, 计算文件系统能支持的最大文件。
- 简洁公式:
 - 每个索引块指针数 (Pointers per block, 记为 K) = $\frac{\text{块大小 Block Size}}{\text{指针大小 Pointer Size}}$
 - 最大文件大小 Max Size = $\text{Block Size} \times (\text{直接指针数} + K + K^2 + K^3)$
- 参考真题: 2025 Spring Q4.1, 2025 Fall Q5.1, 2024 Spring Q4.1, 2024 Fall Q4.1

2. 文件分配所需的总块数 (Total Blocks Required for Allocation)

- 计算目标: 在连续分配或索引分配下, 存储特定大小的文件需要多少个磁盘块。
- 简洁公式:
 - 数据块数 (Data Blocks) = $\lceil \frac{\text{文件大小 File Size}}{\text{块大小 Block Size}} \rceil$
 - 连续分配 (Contiguous): 总块数 = Data Blocks
 - 索引分配 (Indexed): 总块数 = Data Blocks + Index Blocks, 其中 Index Blocks = $\lceil \frac{\text{Data Blocks}}{K} \rceil$
- 参考真题: 2025 Fall Q1.MC14, 2023 Fall Q1.MC18

3. 目录支持的最大文件数 (Max Files in a Directory)

- 计算目标: 在将目录视为文件的系统中, 计算一个目录最多能存多少个文件。
- 简洁公式:
 - 目录项大小 (Entry Size) = 文件名长度 Name Length + 指针大小 Pointer Size
 - 最大文件数 (Max Files) = $\lfloor \frac{\text{系统支持的最大文件大小 Max File Size}}{\text{目录项大小 Entry Size}} \rfloor$
- 参考真题: 2025 Spring Q4.1(b)

4. FAT (文件分配表) 文件大小反推

- 计算目标: 根据 FAT 表的位数和磁盘总容量, 计算块大小及文件大小。
- 简洁公式:
 - 最大块数 Max Blocks = $2^{\text{FAT Entry Bits}}$
 - 块大小 Block Size = $\frac{\text{磁盘总容量 Disk Capacity}}{\text{最大块数 Max Blocks}}$
 - 文件大小 File Size = 占用的条目数 Entries $\times$ 块大小 Block Size
- 参考真题: 2024 Spring Q1.MC19

三、存储与磁盘调度 (Mass Storage & Disk Scheduling)

1. 磁盘有效带宽与传输速率 (Effective Bandwidth / Transfer Rate)

- 计算目标: 计算读写特定大小的文件时的实际磁盘传输带宽。
- 简洁公式:

- 平均旋转延迟 (Avg Rotational Latency) = $0.5 \times \frac{60}{\text{转速 RPM}} \text{ 秒}$
- 单次 I/O 平均时间 (Avg Access Time) = 寻道时间 Seek Time + 平均旋转延迟 + 控制器开销 Controller Overhead
- 传输时间 (Transfer Time) = $\frac{\text{文件大小 File Size}}{\text{驱动器传输速率 Transfer Rate}}$
- 总耗时 Total Time = (占用柱面数/跨柱面数 Cylinders $\times$ Avg Access Time) + Transfer Time
- 有效带宽 (Effective Bandwidth) = $\frac{\text{文件大小 File Size}}{\text{总耗时 Total Time}}$
- 参考真题: 2025 Fall Q5.2, 2025 Spring Q1.MC13, 2024 Spring Q4.3

2. 磁盘调度算法移动距离 (Disk Scheduling Seek Distance)

- 计算目标: 根据不同算法, 计算磁盘臂移动的总柱面数 (Total Seek Distance)。
- 核心算法规则:
 - **FCFS**: 按队列顺序依次移动 (First-Come, First-Served)。
 - **SSTF**: 每次选择离当前磁头最近的请求 (Shortest Seek Time First)。
 - **SCAN**: 朝一个方向移动, **走到磁盘尽头 (如 0 或 max) 后折返**。
 - **C-SCAN**: 朝一个方向移动, **走到磁盘尽头** 后, 直接飞回另一端尽头 (飞回的距离计入总数), 不服务途经请求, 然后继续单向移动。
 - **LOOK**: 朝一个方向移动, **走到该方向最远的请求后折返** (不走到尽头)。
 - **C-LOOK**: 朝一个方向移动, **走到该方向最远的请求后**, 直接飞回另一端最远的请求, 继续单向服务。
- 简洁公式: 总距离 = $\sum |\text{当前位置} - \text{下一个位置}|$
- 参考真题: 2025 Fall Q5.3, 2025 Spring Q4.2, 2024 Spring Q4.4